

Report of Assignment – 3

Course: MA6106 (LLM)

Submitted by: Group 6

- 1. Roland zohmingthana(2611MC03)**
- 2. Harsh Yadav(2611MC04)**
- 3. Achyut Prabhakar(2611MC05)**
- 4. Nikhil Chauhan(2611MC10)**

1. TASK 1

This task implements a Fully Connected Neural Network (FCNN) and backpropagation from scratch using Python/NumPy for two two-dimensional, three-class synthetic datasets. The first dataset contains compact, linearly separable Gaussian-like clusters, while the second contains interleaving spiral arms that require nonlinear decision boundaries.

2. Objectives

- Generate both synthetic datasets with 500 examples per class.
- Split each class into 60% training, 20% validation, and 20% test data.
- Use one hidden layer for the Gaussian dataset and two hidden layers for the spiral dataset.
- Implement stochastic gradient descent and backpropagation from scratch using squared error.
- Test multiple hidden-node configurations and select the strongest architecture using validation performance.
- Present error curves, decision regions, confusion matrices, node-output visualizations, baseline comparison, and inferences.

3. Dataset Generation and Preprocessing

3.1 Gaussian Dataset

The dataset contains 1,500 two-dimensional examples, with 500 examples in each of three classes. The clusters are compact and approximately Gaussian in shape, with different means/covariances. Their arrangement makes the classification problem linearly separable.

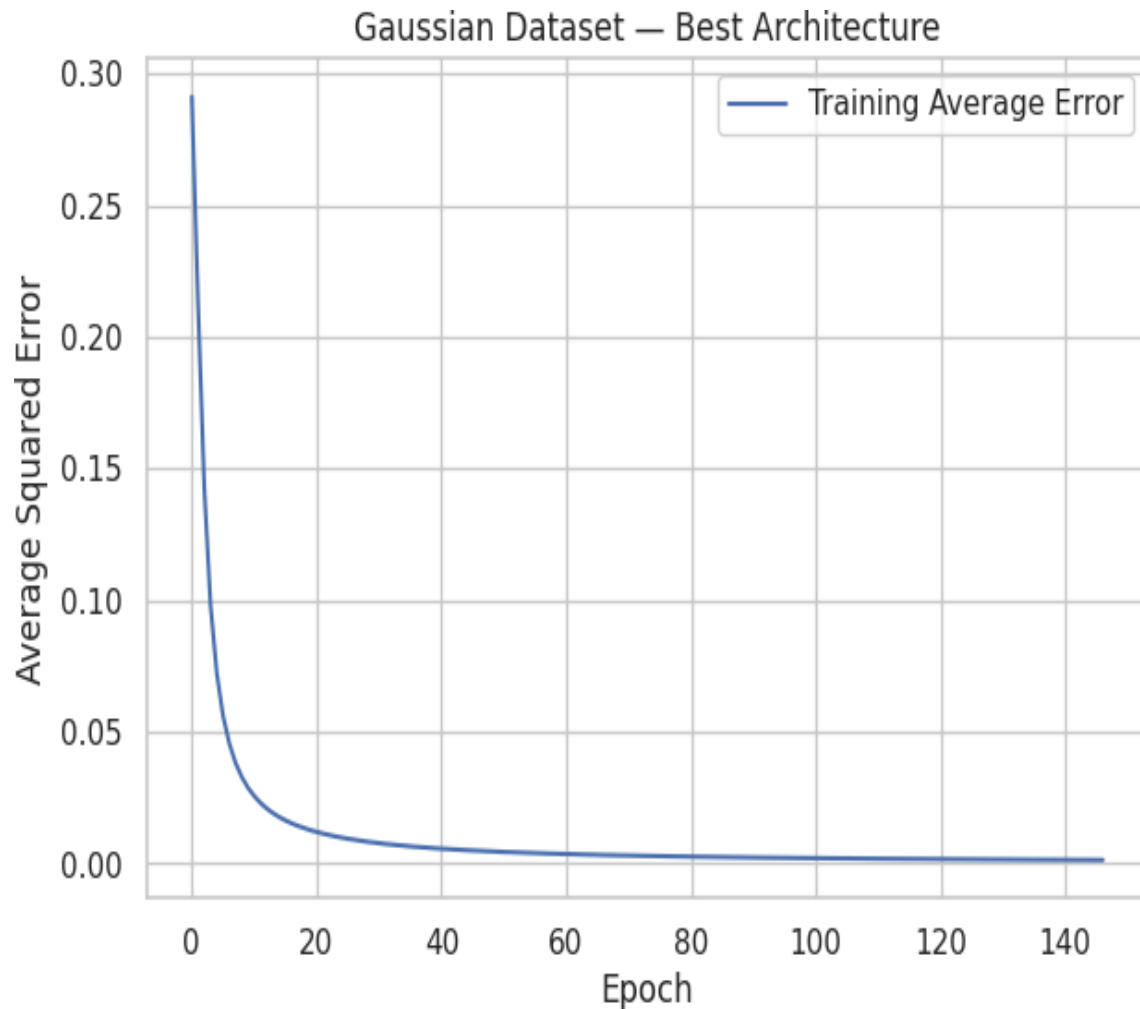


Figure 1. Three-class Gaussian-cluster dataset.

3.2 Spiral Dataset

The dataset also contains 1,500 two-dimensional examples, with 500 examples per class. The three classes form interleaving spiral arms. A single linear decision boundary cannot separate the classes, making this dataset useful for demonstrating the benefit of hidden layers.

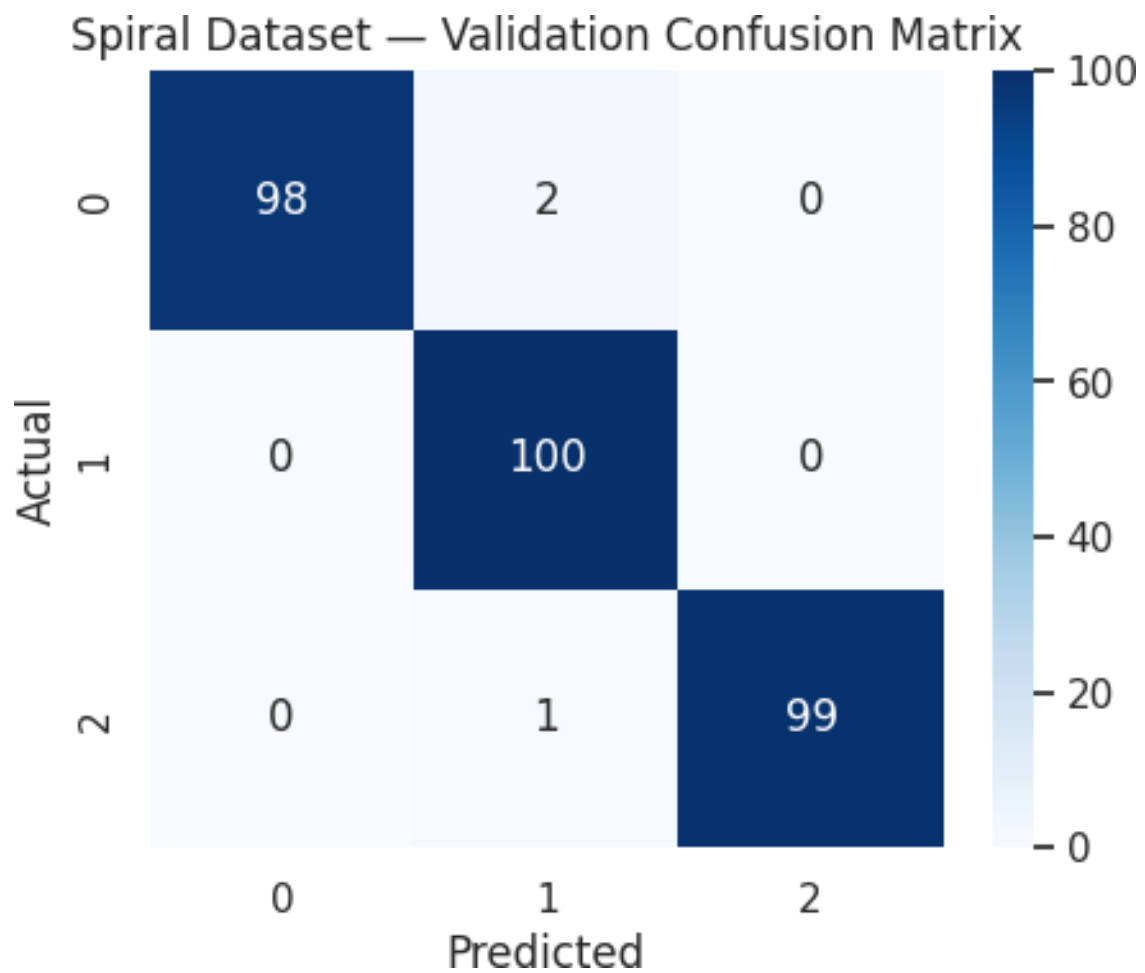


Figure 2. Three-class spiral dataset.

3.3 Data Split and Standardization

For each class, 60% of the examples were assigned to training, 20% to validation and 20% to test. Therefore, each dataset contains 900 training examples, 300 validation examples and 300 test examples, with all three classes represented in every split.

4. FCNN Methodology

The FCNN is implemented directly with NumPy. No high-level neural-network training framework is required for Task 1.

4.1 Network Structure

The input layer has two neurons corresponding to x_1 and x_2 . Hidden neurons use the sigmoid activation function. The output layer has three sigmoid neurons, one for each class, and the predicted class is the index of the largest output activation.

4.2 Forward Propagation

For each input example, activations are propagated from the input layer through the hidden layers to the three output neurons. The sigmoid activation is used in the hidden and output layers.

4.3 Loss Function

The instantaneous loss is the squared error, $E = 0.5 \sum (y - \hat{y})^2$. The training curve reports the average squared error over all training examples in each epoch.

4.4 Backpropagation and SGD

For each training example, the network performs a forward pass, computes the squared error, propagates the error backward through all layers, and immediately updates the weights and biases. This one-example-at-a-time update is stochastic gradient descent.

5. Architecture Experiments

5.1 Gaussian Architectures

5.2 Spiral Architectures

Dataset	Candidate hidden architectures
Gaussian	[2], [4], [8], [16], [32]
Spiral	[4,4], [8,8], [16,8], [16,16], [32,16]

6. Best-Model Validation and Test Results

6.1 Gaussian Dataset

Validation performance for all candidate architectures

Architecture	Validation accuracy	Observation
[2]	99.00%	Smallest tested model.
[4]	99.67%	Best validation result; selected architecture.
[8]	99.67%	No improvement over [4].
[16]	99.67%	No improvement over [4].
[32]	99.67%	No improvement over [4].

Validation confusion matrix — selected [4] architecture

Actual \ Predicted	0	1	2
0	100	0	0
1	0	100	0
2	0	1	99

Validation accuracy: 99.67%.

Test confusion matrix — selected [4] architecture

Actual \ Predicted	0	1	2
0	100	0	0
1	0	100	0
2	0	0	100

Test accuracy: 100.00%.

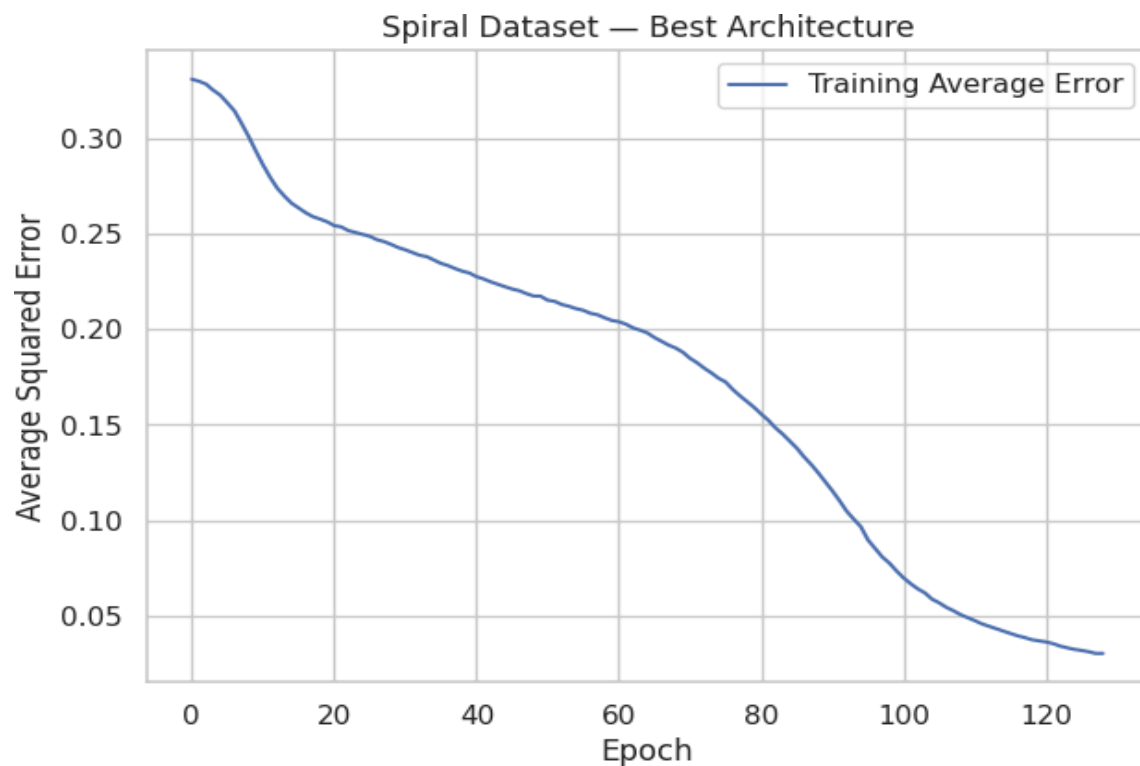
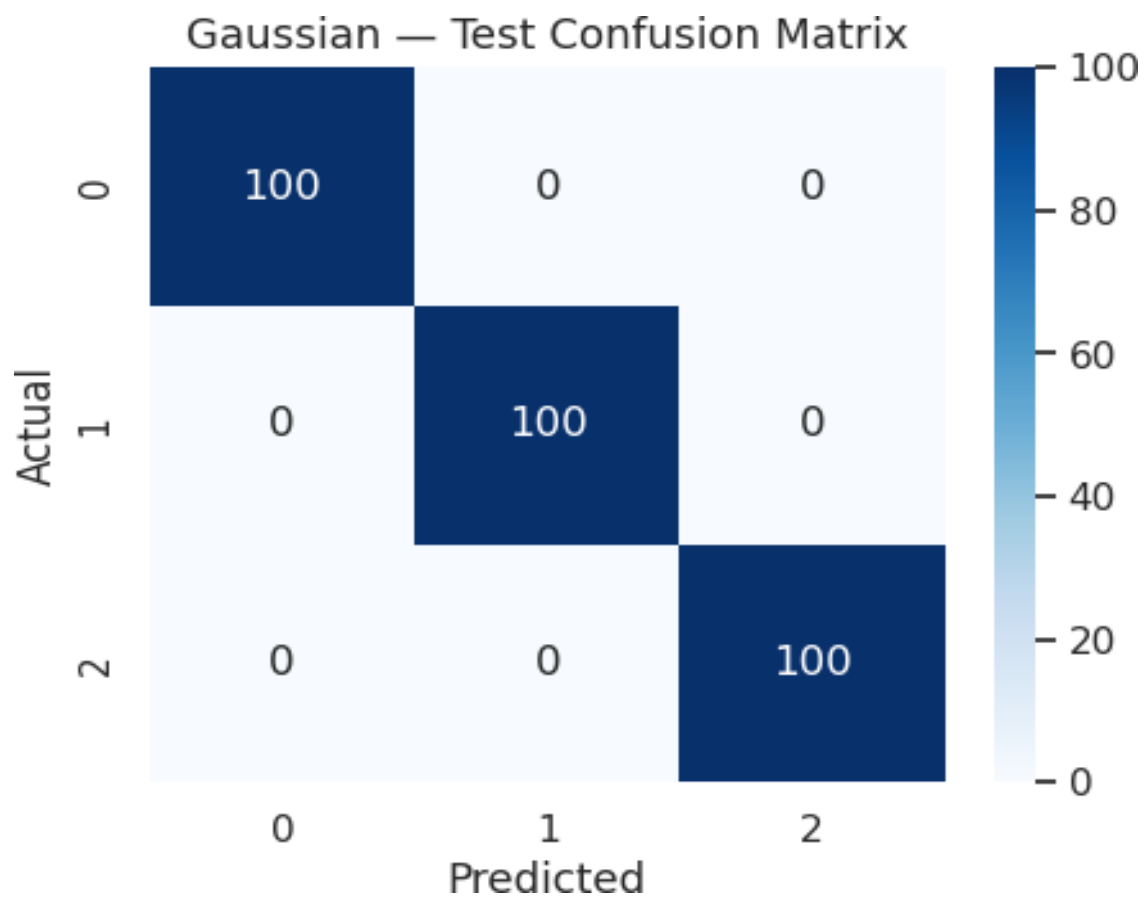


Figure 3. Average squared error versus epochs for the selected Gaussian model.



7. Decision Regions

Figure 4. Gaussian decision regions with training examples.

6.2 Spiral Dataset

Validation performance for all candidate architectures

Architecture	Validation accuracy	Observation
[4,4]	99.33%	Smallest tested two-hidden-layer model.
[8,8]	100.00%	Best validation result; selected architecture.
[16,8]	100.00%	Same accuracy as [8,8].
[16,16]	100.00%	Same accuracy as [8,8].
[32,16]	100.00%	Same accuracy as [8,8].

Validation confusion matrix — selected [8,8] architecture

Actual \ Predicted	0	1	2
0	100	0	0
1	0	100	0
2	0	0	100

Validation accuracy: 100.00%.

Test confusion matrix — selected [8,8] architecture

Actual \ Predicted	0	1	2
0	100	0	0
1	0	100	0
2	0	0	100

Test accuracy: 100.00%.

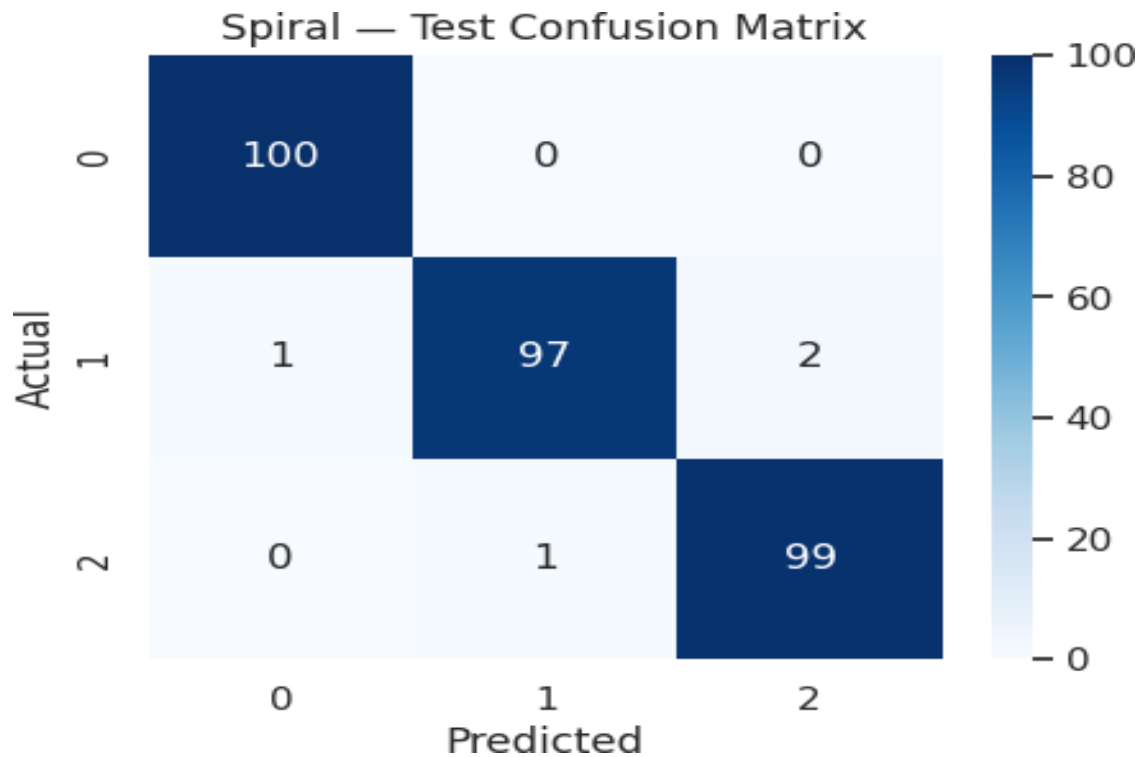


Figure 5. Average squared error versus epochs for the selected Spiral model.

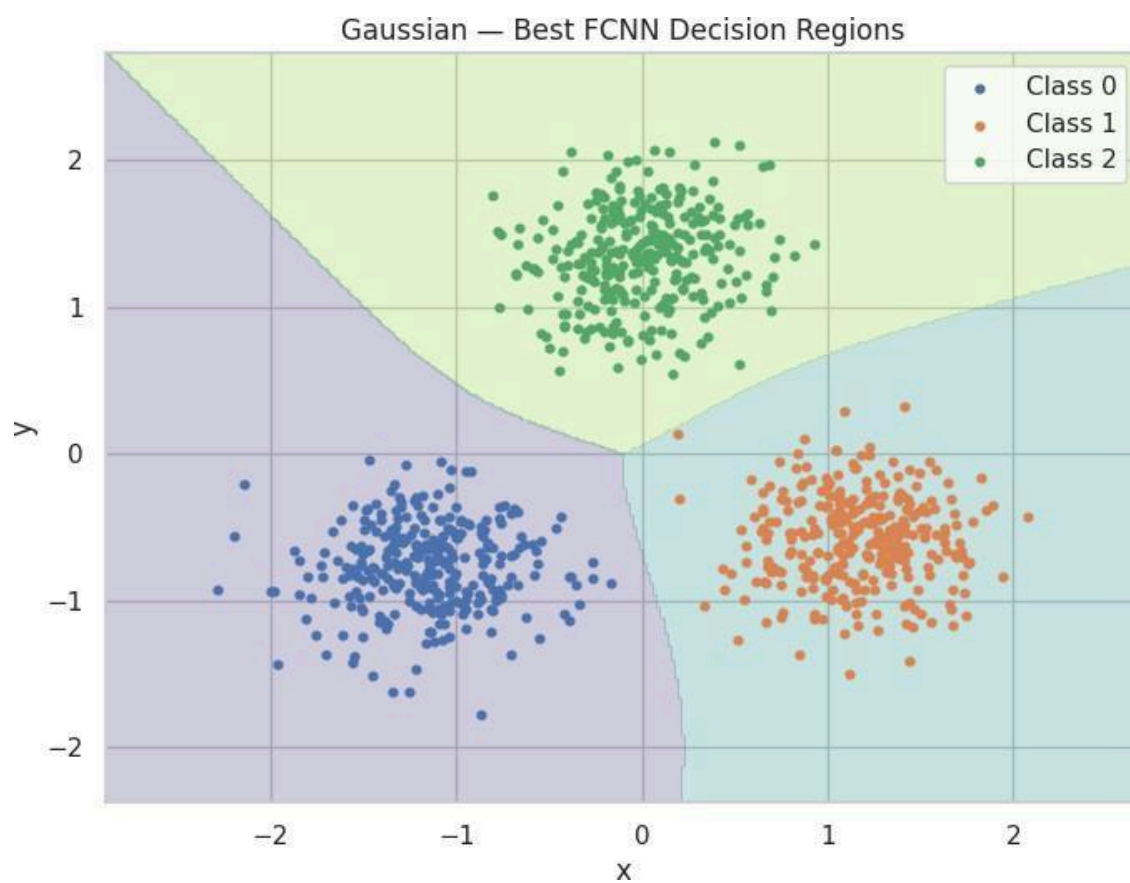


Figure 6. Spiral decision regions with training examples.

8. Internal Node-Output Visualizations

For the best architecture of each dataset, the notebook generates 3-D plots for every hidden/output node. The horizontal axes are the original input variables x_1 and x_2 , while the vertical axis is the activation of the selected node. These plots are produced separately for training, validation and test examples.

The complete set contains many individual node plots. Representative examples are shown below; the cleaned notebook retains the full train/validation/test generation procedure for every node.

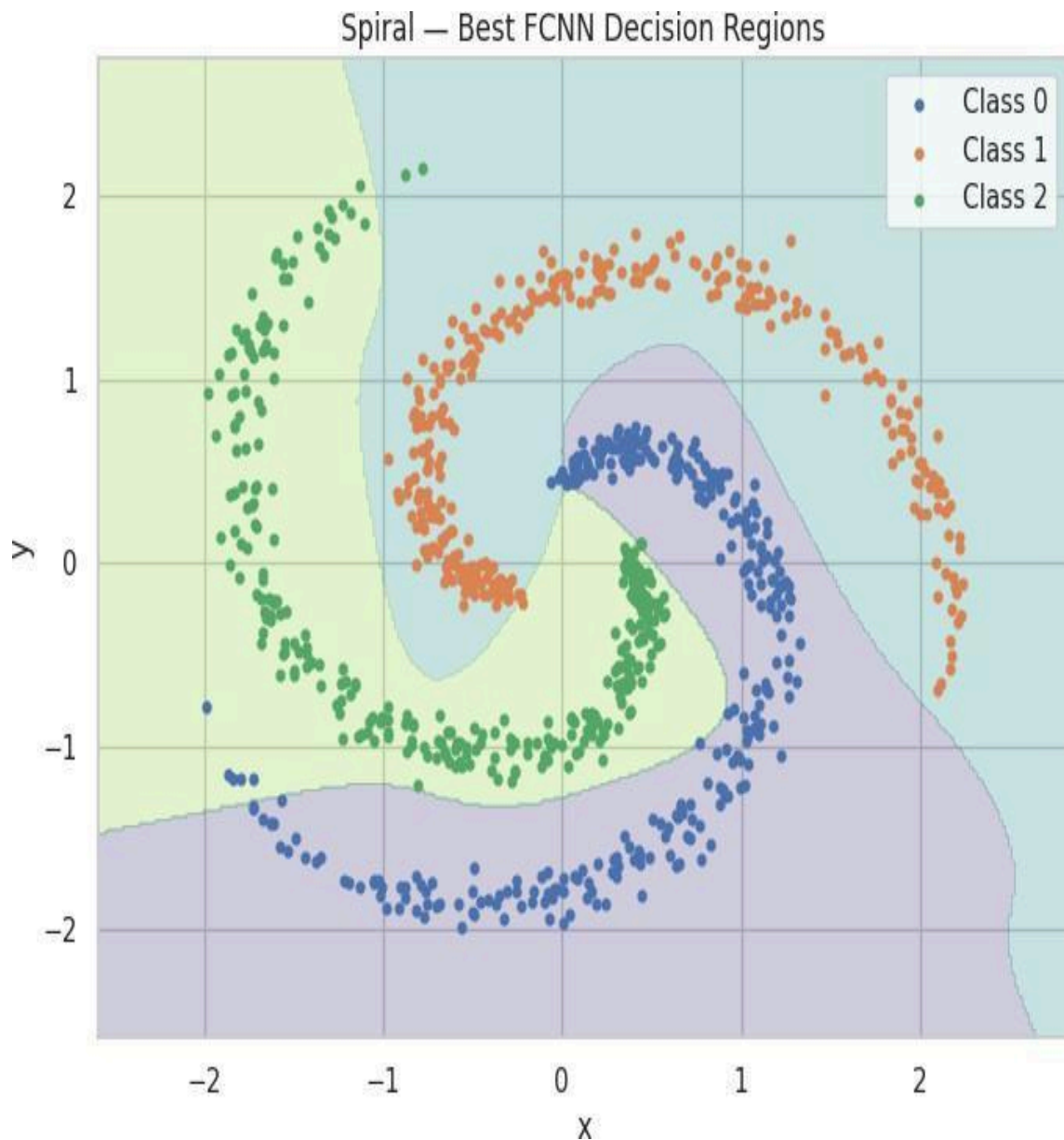


Figure 7. Representative Gaussian hidden-node activation surface.

Gaussian | Training | Layer 1, Node 1

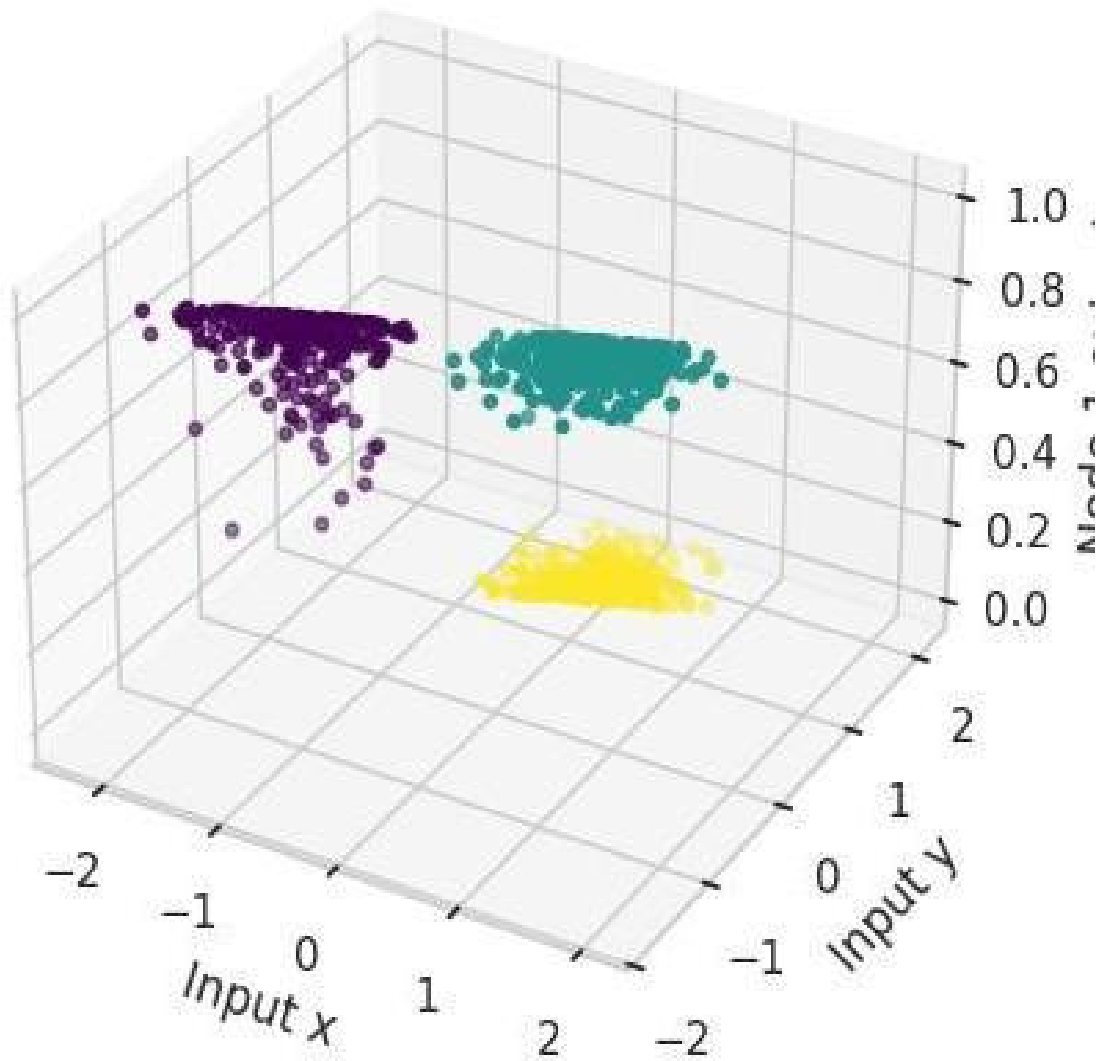


Figure 8. Representative Gaussian output-node activation surface.

Spiral activation surfaces

Gaussian | Training | Layer 1, Node 3

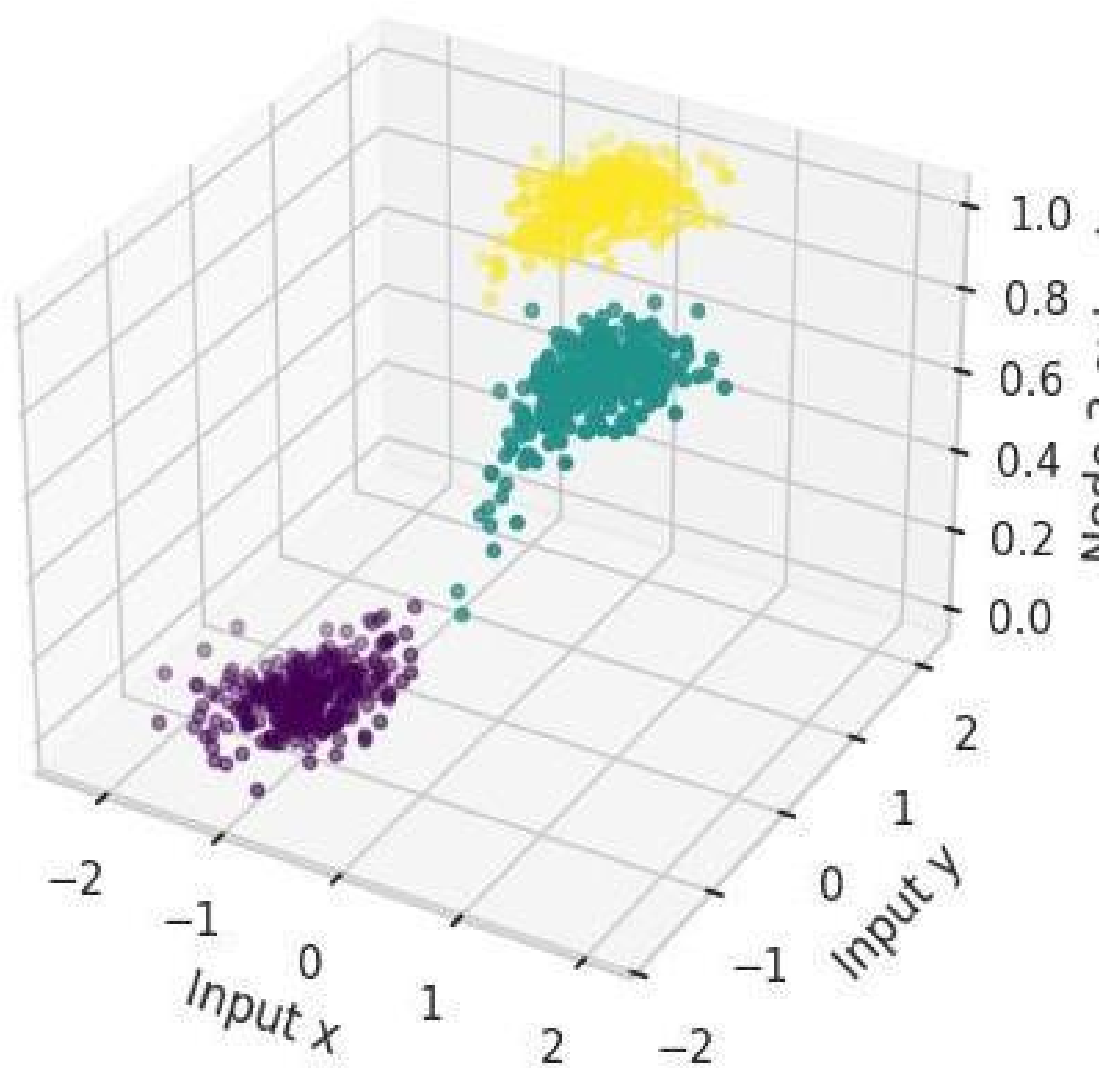


Figure 9. Representative Spiral hidden-layer activation surface.

Spiral | Training | Layer 1, Node 1

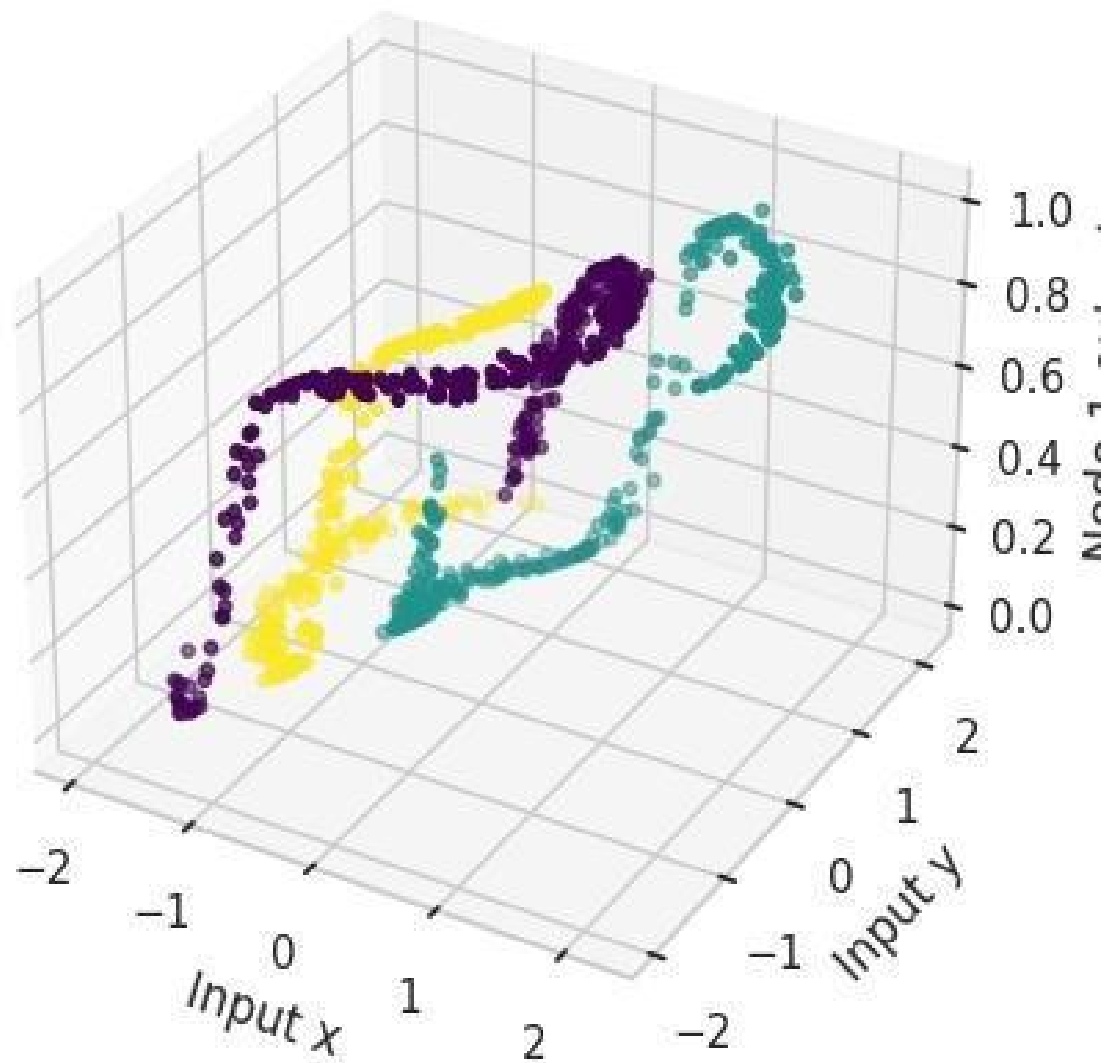


Figure 10. Representative Spiral test activation surface.

9. Single-Neuron Baseline

The comparison model has the same two input variables and three output classes but no hidden layer. It therefore represents a linear baseline.

Dataset	Model	Validation	Test
Gaussian	No hidden layer	≈100%	≈100%
Gaussian	Best FCNN [4]	99.67%	100.00%
Spiral	No hidden layer	≈52%	≈52%
Spiral	Best FCNN [8,8]	100.00%	100.00%

Interpretation: the Gaussian problem is already linearly separable, so a hidden layer provides little practical benefit. In contrast, the spiral problem cannot be solved with a single linear boundary, and the two-hidden-layer FCNN improves the result dramatically.

10. Inferences and Discussion

1. Effect of model capacity

For the Gaussian dataset, increasing the hidden-layer size beyond four nodes did not improve validation accuracy. This indicates that the classification problem is simple enough for a small network.

2. Nonlinearity is important for the spiral dataset

The spiral dataset provides the clearest evidence for the usefulness of hidden layers. The no-hidden-layer baseline performs poorly, whereas a two-hidden-layer network reaches 100% validation and test accuracy in the experiment.

3. Decision-region interpretation

For the Gaussian data, the learned regions are simple and approximately linear, matching the geometry of the clusters. For the spiral data, the learned regions curve around the spiral arms, showing that the hidden layers have learned a nonlinear transformation of the inputs.

4. Error curves

The average squared-error curves show how the model learns over epochs. A decreasing curve indicates that the network is reducing the training loss. The final architecture is preferred not merely because the training error is low, but because its validation/test classification performance is strong.

5. Activation surfaces

The hidden-node surfaces demonstrate that each hidden neuron maps the original two-dimensional input into a new nonlinear feature. The output neurons then combine these learned features to produce class-specific responses.

6. Overall conclusion

The experiment demonstrates the central advantage of an FCNN: hidden layers are not especially necessary for an already linearly separable problem, but they are crucial when the class boundaries are nonlinear.

11. Conclusion

Dataset	Selected architecture	Validation	Test
Gaussian	[4]	99.67%	100.00%
Spiral	[8,8]	100.00%	100.00%

TASK 2:

1. Objectives

- Implement an FCNN for five-class Fashion-MNIST classification.
- Create a balanced dataset containing the five selected categories.
- Compare FCNNs with three, four, and five hidden layers.
- Evaluate SGD, Batch Gradient Descent, Momentum, NAG, RMSProp, and Adam.
- Examine the convergence behavior of the different optimizers.
- Compare training and validation accuracy across all architecture–optimizer pairs.
- Select the strongest model using validation accuracy.
- Assess the selected model on unseen test data using accuracy and confusion matrices.

2. Dataset and Preprocessing

2.1 Fashion-MNIST Dataset

The Fashion-MNIST data used in the experiment was loaded through the torchvision implementation. The original dataset consists of 60,000 training images and 10,000 test images, with each image represented as a 28×28 grayscale image. Five categories were selected for the present study. Each selected category contributes 7,000 samples, giving a balanced dataset of 35,000 images.

Label	Class
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat

2.2 Data Split

A stratified partition was applied so that the class proportions remained identical across the training, validation, and test subsets. The final split contained 22,400 training images, 5,600

validation images, and 7,000 test images. Every subset remained perfectly balanced across the five classes.

Subset	Samples	Percentage
Training	22,400	64%
Validation	5,600	16%
Test	7,000	20%
Total	35,000	100%

2.3 Input Representation

Each image contains $28 \times 28 = 784$ pixels. Consequently, the image is flattened into a 784-dimensional vector before being supplied to the FCNN. Pixel intensities are normalized to the interval $[0, 1]$, producing an input representation of $35,000 \times 784$ for the complete selected dataset.

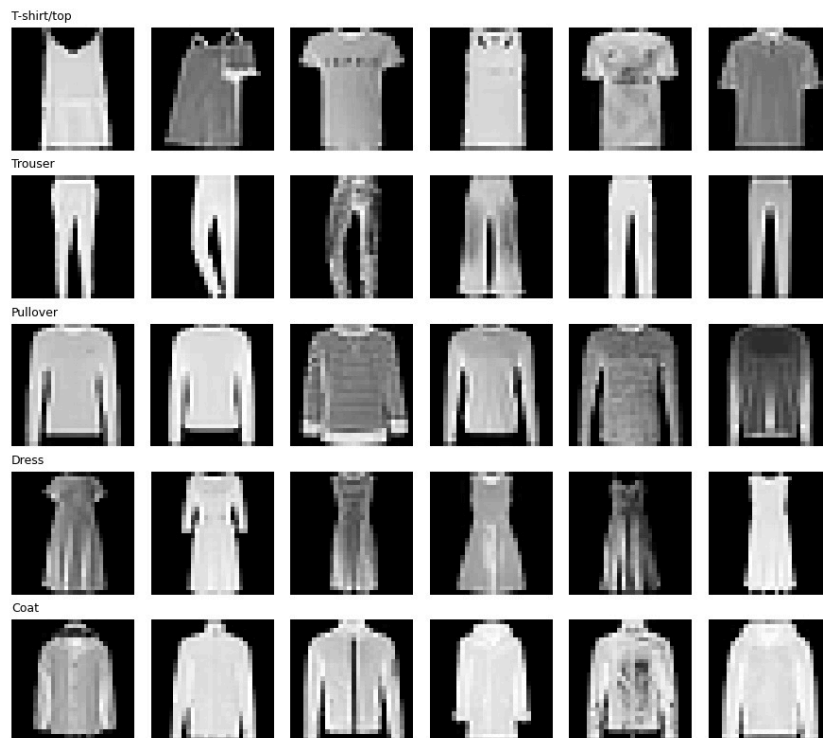


Figure 1. Representative training samples from the selected Fashion-MNIST classes.

3. Methodology

3.1 Fully Connected Neural Network Design

The classifier uses a fully connected architecture with 784 input neurons, reflecting the flattened image representation. ReLU activation is applied in the hidden layers, while the final layer contains five output neurons, one for each target class. Three network depths were investigated.

Architecture	Hidden-layer structure	Parameters
3-hidden	128 - 64 - 32	1,10,981
4-hidden	256 - 128 - 64 - 32	2,44,357
5-hidden	256 - 128 - 128 - 64 - 32	2,60,869

The architecture that ultimately performed best had the structure 256 - 128 - 128 - 64 - 32. Each hidden layer computes a linear transformation followed by ReLU activation, while the output layer produces class scores.

3.2 Loss Function

Cross-entropy was used as the objective function. In the implementation, this loss was realized through PyTorch's `CrossEntropyLoss`, with the target labels compared against the predicted class probabilities.

3.3 Optimization Methods

Six optimization approaches were compared under the experimental settings reported in the original notebook/report:

Optimizer	Configuration
SGD	Learning rate $\alpha = 0.001$; batch size = 1
Batch Gradient Descent	Entire training set used for each update
Momentum	$\alpha = 0.001$; $\mu = 0.9$
NAG	$\alpha = 0.001$; $\mu = 0.9$
RMSProp	$\alpha = 0.001$; $\beta = 0.99$; $\varepsilon = 10^{-8}$
Adam	$\alpha = 0.001$; $\beta_1 = 0.9$; $\beta_2 = 0.999$; $\varepsilon = 10^{-8}$

3.4 Initialization and Convergence

A random seed of 7 was fixed for reproducibility. For each architecture, the same saved initial weights were reused when comparing optimizers, reducing the effect of different starting points. Training was allowed to continue for up to 100 epochs. Convergence was declared when the absolute change in average training error between consecutive epochs fell below 0.001, with convergence checks beginning after the fifth epoch. This design produced $3 \times 6 = 18$ experiments.

4. Experimental Results and Analysis

4.1 Overall Training and Validation Performance

The 18 experiments produced noticeably different results. The **five-hidden-layer model with Momentum** ranked first, with 94.55% training accuracy and 91.61% validation accuracy after 27 epochs. The **three-hidden-layer model with Adam** was a close second, while the three-hidden-layer model with Adam achieved slightly higher training accuracy but lower validation accuracy. At the opposite end, Batch Gradient Descent remained close to the 20% chance level expected for five balanced classes.

Architecture	SGD	Batch GD	Momentum	NAG	RMSProp	Adam
arch1 (128-64-32)	90.49	7.91	94.25	92.95	95.14	95.16
arch2 (256-128-64-32)	91.39	20.00	92.79	93.60	94.36	95.91
arch3 (256-128-128-64-32)	91.62	20.00	94.55	94.67	94.01	94.80

Table : Training accuracy (%)

Architecture	SGD	Batch GD	Momentum	NAG	RMSProp	Adam
arch1 (128-64-32)	89.39	7.55	91.36	90.57	91.16	91.48
arch2 (256-128-64-32)	89.98	20.00	90.11	90.71	91.11	91.09
arch3 (256-128-128-64-32)	89.93	20.00	91.61	91.20	91.02	91.05

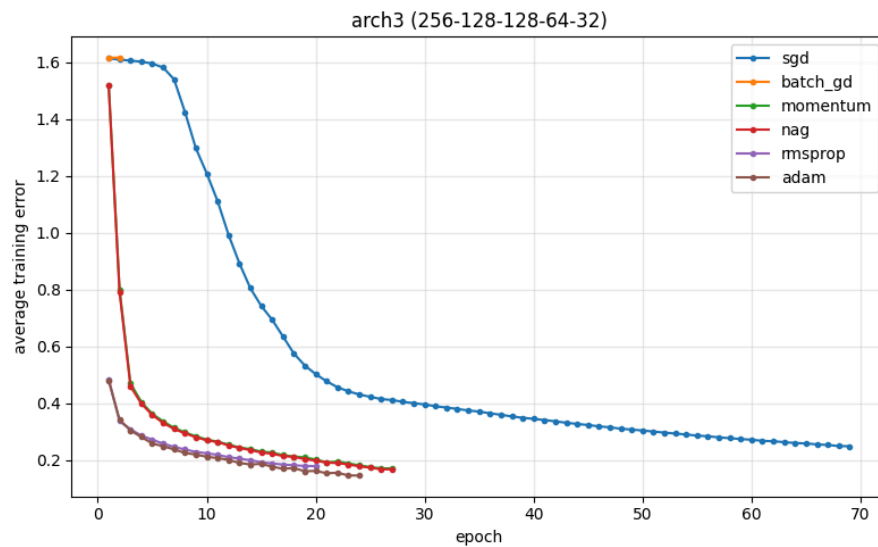
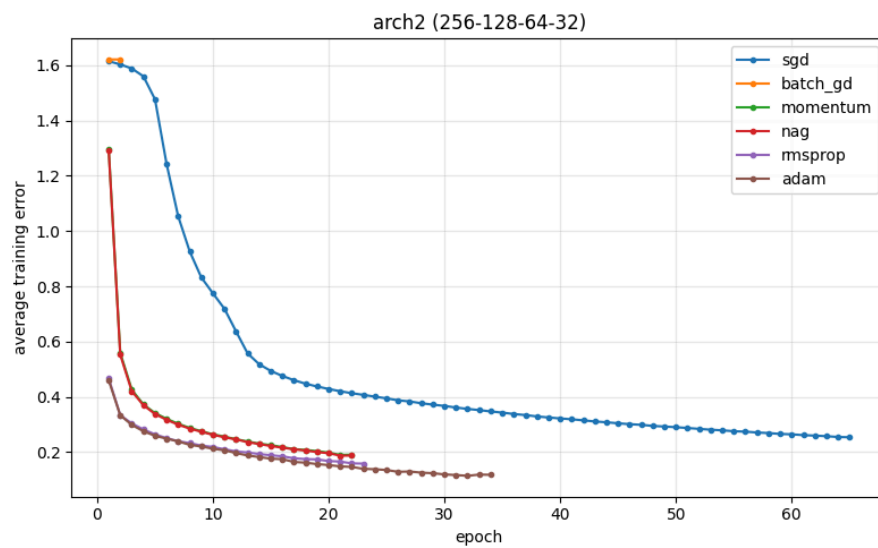
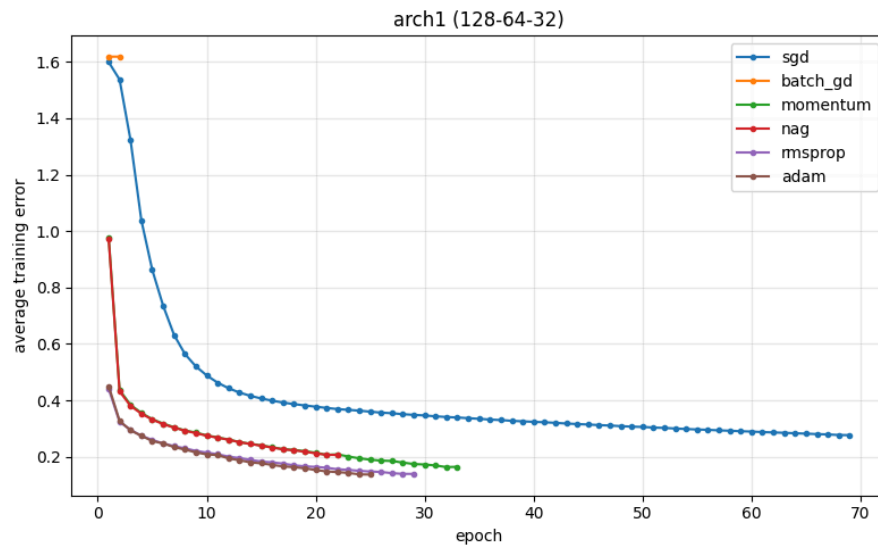
Table : Validation accuracy (%)

4.2 Convergence Behavior

Architecture	SGD	Batch GD	Momentum	NAG	RMSProp	Adam
arch1 (128-64-32)	69	2	33	22	29	25
arch2 (256-128-64-32)	65	2	22	22	23	34
arch3 (256-128-128-64-32)	69	2	27	27	20	24

Table: Epochs to convergence

4.3 Interpretation of Training Error Curves



Adam and RMSProp fall fastest. They start at about 0.45 after epoch 1, reach about 0.2 by epoch 10, and finish lowest (about 0.12 to 0.17). **Momentum and NAG** start much higher (about 1.0 in arch1, 1.3 in arch2 and 1.5 in arch3) but catch up within about 5 epochs. Their curves almost overlap, because the look-ahead step in NAG changes little at $lr = 0.001$. **Plain SGD** is by far the slowest. It decreases gradually and is still sloping downward at epoch 65 to 69, ending at about 0.25 to 0.27, well above the others. **Batch GD** stops after 2 epochs at about 1.6, which is $\ln(5)$ (random guessing among 5 classes). It made only 2 weight updates, so the stopping rule ended it, not convergence.

Fewer epochs doesn't mean a better model. The stopping rule fires when the change between epochs drops below $10e-4$, not at a minimum, and most curves are still slowly decreasing when they stop.

The ordering among Momentum, NAG, RMSProp and Adam changes between architectures, so we can't claim which one is fastest from this data.

Training error and validation accuracy tell different stories. SGD has the highest final error but about 90% validation accuracy, only 1 to 2 points below the others, so we have to compare both.

4.4 Best Architecture and Optimizer

The highest validation accuracy was obtained by the five-hidden-layer FCNN trained with Momentum. Its training accuracy was 94.55%, its validation accuracy was 91.61%, and it converged in 27 epochs. Therefore, the selected model for final testing was the five-hidden-layer FCNN with Momentum.

4.5 Test-Set Evaluation

After model selection based on validation performance, the chosen network was evaluated on the unseen test set. It achieved 90.36% test accuracy. The close relationship between validation and test performance suggests that the selected model transferred its learned decision boundaries reasonably well to previously unseen images.

Metric	Accuracy
Training	94.55%
Validation	91.61%
Test	90.36%

4.6 Confusion-Matrix Analysis

The confusion matrices provide a class-by-class view of the model's predictions. They make it possible to identify which clothing categories are recognized reliably and which categories are more frequently confused. The supplied report presents separate matrices for the training and test sets.

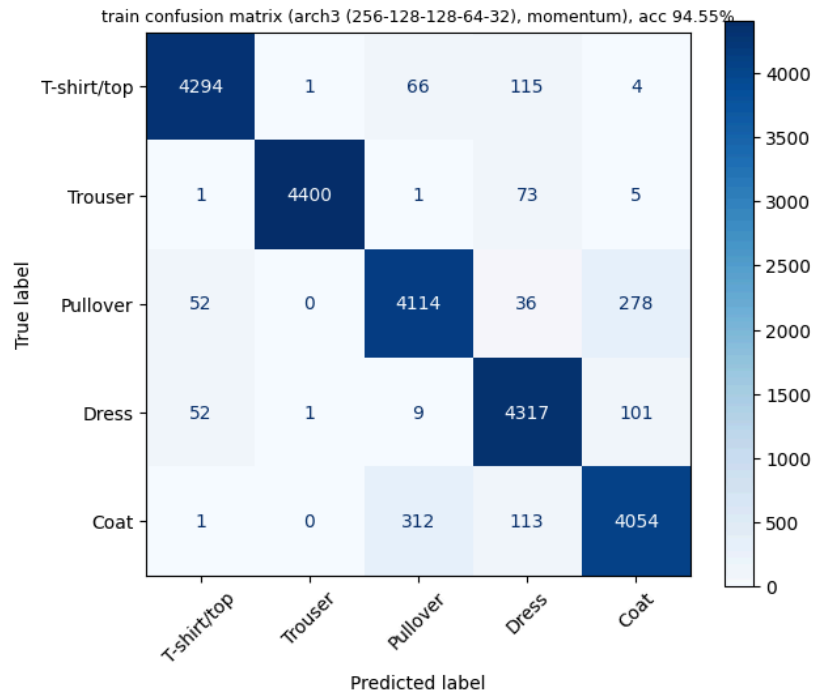


Figure 3. Confusion matrix for the training

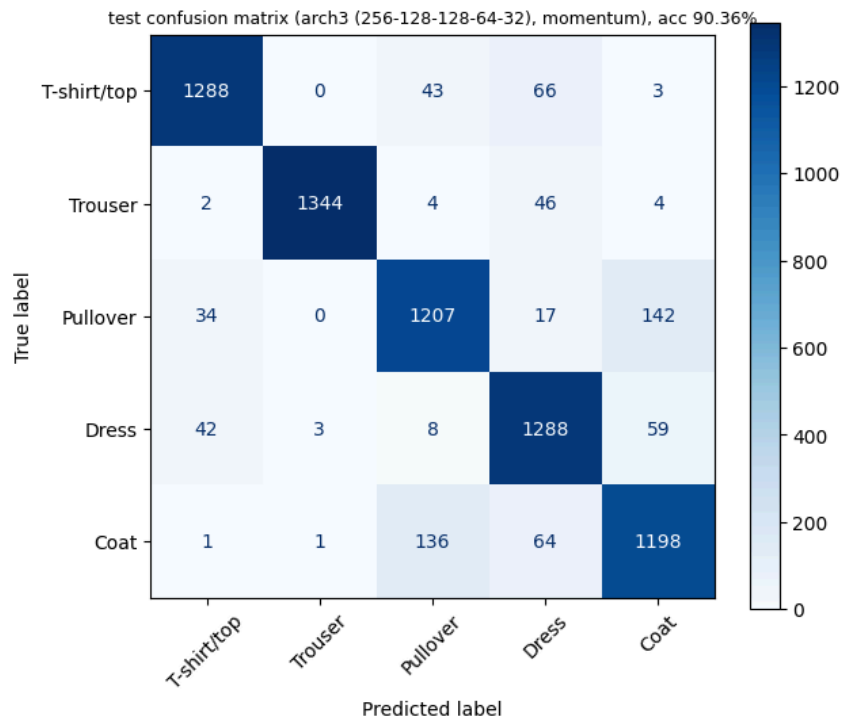


Figure 4. Confusion matrix for the test set.

4.7 Comparison of Network Depth

When Momentum is held constant, the five-hidden-layer model provides the best validation result. Moving from three to four hidden layers reduces validation accuracy from 91.36% to 90.11%. Adding a fifth hidden layer increases validation accuracy to 91.61%. In other optimizers such as rmsprop and adam validation accuracy decreases when you add a hidden layer.

Architecture	Training Accuracy	Validation Accuracy
3-hidden	94.25%	91.36%
4-hidden	92.79%	90.11%
5-hidden	94.55%	91.61%

4.8 Comparison of Optimizers

Across the three architectures, 5 Hidden layer model with Momentum achieved the highest validation score among the tested optimizers. Adam and RMSProp followed closely, while SGD and NAG performed somewhat lower. Batch Gradient Descent was substantially weaker, with a best validation accuracy of only 20.00%.

Optimizer	Best Validation Accuracy
SGD	89.98%
NAG	90.71%
Momentum	91.61%
Adam	91.48%
RMSProp	91.16%
Batch GD	20.00%

Class	Test accuracy (%)	Why?
T-shirt/top	92.00	Good. Likely gets confused with Pullover or Dress at times.
Trouser	96.00	Easiest class. Its two-legged shape looks different from every other class.
Pullover	86.21	One of the two weakest . It looks very similar to Coat. .
Dress	92.00	Good. Some overlap with T-shirt/top is plausible
Coat	85.57	Weakest. Long sleeves and a similar outline to Pullover.
Average (overall)	90.36	Trouser is the easiest to predict and Pullover and Coat somewhat harder

5. Conclusion

A fully connected neural network was successfully evaluated for five-class Fashion-MNIST image classification. The experiment compared three network depths with six optimization algorithms, resulting in 18 training configurations. The dataset contained 35,000 balanced images and was divided into 64% training, 16% validation, and 20% test data. The five-hidden-layer FCNN trained with Momentum produced the highest validation accuracy of 91.61% and achieved 90.36% on the final test set.

Overall, the experiment demonstrates that a comparatively simple FCNN can classify the selected Fashion-MNIST categories effectively. The choice of optimizer had a major impact on learning quality. Under the reported experimental settings, five hidden layers combined with Momentum offered the strongest overall performance.

We can't say the five-hidden layer with momentum will give best results on some other dataset. We tested 5 Fashion-MNIST classes, three architectures, one learning rate (0.001), a single run each, and one stopping rule. A different dataset, model or learning rate can change the ranking. Every optimizer needs its own good learning rate. At 0.001, Adam and RMSProp are near their usual defaults, while plain SGD and Batch GD usually need larger rates, so the comparison favoured the adaptive methods. Batch size and stopping rule also matters.

On this dataset and under these settings, five-hidden-layer withh Momentum gave the highest validation accuracy, although the difference from NAG, RMSProp and Adam was within what a different random seed could change. These results may not carry over to other datasets, architectures or learning rates.